EN.705.643 Final Project Proposal

Finding Climbable Rock Using Airborne Laser Scans

Zachary Freund Yixian Li

1 Introduction & Motivation

Rock climbers find new cliffs by word of mouth and by walking. Websites list the climbs that came out of that process, but nothing lists the cliffs nobody has looked at. What if a model could predict whether a location has climbable rock? Climbers could find new routes without blindly wandering around, and a similar model could identify cliff habitat for birds of prey for wildlife conservation.

LiDAR (light detection and ranging) scanners fire laser pulses from an aircraft and time their reflections, producing a point cloud: millions of 3D points mapping the shape of the terrain. Most of the United States has been scanned this way, and the scans are freely available through the U.S. Geological Survey (USGS) [1]. Here we ask whether a deep learning model can pick out climbable cliffs in those scans, and in particular whether it needs the raw 3D points or whether a simple top-down height map is enough. The point cloud is richer but more computationally expensive. A digital elevation model (DEM) flattens it into an image whose pixel values are ground height, which is smaller and easy to feed to an image network. We will train models on both and compare.

The biggest challenge is that we have to make our own labels. No dataset marks climbable rock in a scan, and the closest substitute, Mountain Project [2], where climbers record existing climbs, is only a rough guide. It gives one coordinate for a whole section of cliff, often 10–30 m from the rock, and it records only where people do climb, so it has no negative examples of cliffs that are unclimbable (e.g., due to rock type). Obvious negatives, such as the flat ground above a cliff and the rubble below it, look nothing like a cliff and make the task easier than it should be. Creating suitable labels is therefore an important part of the project.

2 Related Work

PointNet++ [3] is the deep neural network we will use for 3D input. It takes an unordered set of points, groups nearby points into small neighbourhoods, summarises each one, and repeats this at larger scales. It is usually pretrained on ModelNet40 [4], a collection of 3D models of everyday objects such as chairs and lamps. Li et al. [5] ran PointNet++ and two other 3D deep learning models on airborne scans to label each point as building, vegetation, or ground, beating a classical random-forest classifier.

In terms of using airborne data for detecting terrain features, Albrecht et al. [6] found ancient ruins in airborne scans from few, noisy labels, and Rafique et al. [7] found sinkholes, reporting that the gradient of the DEM works far better than aerial photos. Both flatten the scan to 2D first. That is reasonable for ruins and sinkholes, which are gentle shapes. We argue that cliffs are the case where flattening should hurt most and where the raw 3D point data might give us better results.

3 Data & models

We will gather data for several climbing areas in Arkansas. So far we have run a proof of concept on Horseshoe Canyon Ranch near Jasper; the other areas have not been chosen or downloaded yet.

3.1 Data:

USGS 3DEP scans: Free airborne LiDAR scans from the US Geological Survey's national programme [1]. In the proof of concept: the 2017 survey of the area, 18.66 M points over 1.5 × 1.7 km (7.3 per m²).

Mountain Project: A website where climbers record climbs [2], giving one coordinate per cliff section. In the proof of concept: 530 climbs at 33 coordinates.

Open Climbing:

Application for mapping climbing areas and creating interactive climbing guides[11]. This website is an open source place to record climbing areas and specific route locations. Open Climbing utilizes OpenStreetMap for the map engine backing.

Training examples: Locations we label as climbable or not, each with the scan around it. In the proof of concept: 354 labelled as climbable and 1,093 not, from 30 cliff sections.

3.2 Pretrained models:

For 3D, a PointNet++ [3] checkpoint trained on ModelNet40 [4] (from the open-source yanx27 implementation [8], MIT licence). For 2D, a ResNet-18 [9] trained on ImageNet [10].

3.3 Feasibility:

We ran the proof of concept to assess feasibility. The scan, the climb list and the PointNet++ checkpoint were all downloaded and loaded locally. The model also appeared to run as expected. The pretrain checkpoint loads cleanly, and a two-epoch trial trains and evaluates end to end.

4 Experiments

4.1 Creating the Dataset: Are we labeling correctly?

We will use a small set of precise routes to generate pseudo-labels at scale. To check the heuristic, we measure the distance from each pseudo-label to the true route location and plot these errors as a CDF. We then train three models: precise data only, pseudo-labels only, and pseudo-labels fine-tuned on precise data. If the pseudo-label-only model is far worse, we refine the heuristic before generating the full dataset.

4.2 Determining the Proper Confidence Threshold

We run the model over whole areas and group its predictions around known crags. The threshold on P(Climbable) should make predicted route counts per crag match the true totals (Mean Absolute Percentage Error across dozens of crags), and make prediction density fall off with distance from each crag as true route density does, which filters out false detections.

4.3 Geometric Baseline Comparison

A model could simply learn "vertical surface = climbing route". We compare it with simple geometric rules, such as flagging surfaces steeper than 75°. Beating them shows the model learns what makes rock climbable, not just where cliffs are.

4.4 Comparing 3-D Point Cloud Data to 2.5D Elevation Gradient Model

To justify the cost of 3D, we compare it with a 2D model on two-channel elevation-gradient images with weighted cross-entropy. Both go through the same experiments: robustness to pseudo-label noise, threshold tuning, and the geometric baseline. Their distance errors and clustering metrics show whether raw 3D geometry is needed.

5 Training Plan

5.1 Data scope and splits

Three to four climbing areas. Each input is the column of points within 5 m of a location, resampled to 1,024 points and scaled into a unit sphere as in ModelNet40. Splits are by whole crag or wall, since neighbouring route patches overlap. Two test sets are never used for training or model selection:

Precise test set: about 25% of the precise OpenClimbing routes, scattered across the USA, giving exact ground truth over many rock types. The other 75% are for training and validation.

Known-area test set: a third crag at Horseshoe Canyon Ranch (HCR), with routes we locate by hand, while two main HCR sections are used for training.

5.2 Loss, optimizer, schedule

Loss: cross-entropy weighted by inverse class frequency (about 3 negatives per positive).

Optimizer: AdamW, learning rate 10^{-3} for the head and 10^{-4} for the pretrained backbone so its filters change slowly, weight decay 10^{-4}. We compare scratch, frozen backbone and full fine-tune.

Schedule: cosine annealing over 60 epochs, batch size 32. Early stopping (patience 12) keeps the best validation balanced accuracy, since the small dataset overfits quickly.

5.3 Augmentation

A new random 1,024-point sample on each read, rotation about the vertical axis only (no tilting, as a cliff's angle to gravity matters), scaling by 0.9–1.1 and jitter ($\sigma = 0.01$).

5.4 Metrics

Balanced accuracy and AUROC, which handle class imbalance, plus precision, recall, F1 and the confusion matrix. Accuracy by negative type and recall by LiDAR coverage separate model errors from rock the scanner never saw. Results are averaged over 3 training seeds (mean ± std).

5.5 Compute and time estimate

All local on one RTX 3080 (10 GB), 16 CPU threads, 19 GB RAM. We measured 260 patches/s training and 380 inference, using 2.9 GB of GPU memory. With about 5k patches a run takes 15–20 min, and all 27 runs (3 modes × 3 label settings × 3 seeds) take about 8 h overnight, plus about 1 h for the 2D model.

5.6 Timeline

Phase 1: download data, locate HCR test routes by hand, check pseudo-labels.

Phase 2: train the 3D and 2D models; run the geometric baseline.

Phase 3: tune the threshold, compare 2D and 3D, score both test sets, write up.

6 Collaboration & resources

6.1 GitHub

`https://github.com/euskon/705-643-final-project`

6.2 Hugging Face

`https://huggingface.co/DeepLearningPyTorchF26/FinalProject`

6.3 Collaboration Strategy

Each of us works on our own branch and merges into a shared main branch as needed. Zach trains the 3D PointNet++ models and Yixian trains the 2D ResNet-18 models, each on our own machine, and we both upload checkpoints to the Hugging Face repository.

References

[1] U.S. Geological Survey. 3D Elevation Program (3DEP). https://www.usgs.gov/3d-elevation-program. Accessed Oct. 2026.

[2] Mountain Project. https://www.mountainproject.com. Accessed Oct. 2026.

[3] C. R. Qi, L. Yi, H. Su, L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. NeurIPS, 2017.

[4] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, J. Xiao. 3D ShapeNets: A deep representation for volumetric shapes. CVPR, 2015.

[5] N. Li, O. Kähler, N. Pfeifer. A comparison of deep learning methods for airborne lidar point clouds classification. IEEE JSTARS, 14:6467–6486, 2021.

[6] C. M. Albrecht et al. Learning and recognizing archeological features from LiDAR data. IEEE Big Data, 2019.

[7] M. U. Rafique, J. Zhu, N. Jacobs. Automatic segmentation of sinkholes using a convolutional neural network. Earth and Space Science, 2022.

[8] X. Yan. Pointnet/Pointnet++ Pytorch. https://github.com/yanx27/Pointnet_Pointnet2_pytorch, 2019.

[9] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. CVPR, 2016.

[10] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei. ImageNet: A large-scale hierarchical image database. CVPR, 2009.

[11] Open Climbing https://openclimbing.org. Accessed Oct. 2026.